Compte rendu de travaux pratiques

Programmation embarquée

Prise en main de Keil µVision et commande
d'une broche GPIO du microcontrôleur STM32F407

Enseignant de TP

M. Atef Ftirich

Réalisé par

Bouabdallah Mohamed Yacine
Ben Hmida Ahmed
Arrami Mohamed Ali

Classe : LISI3

Année universitaire : 2026/2027

8 octobre 2026

Sommaire

1. Introduction et objectif 2
2. Présentation de la carte STM32 2
2.1 Le microcontrôleur STM32F407VG 2
2.2 Architecture interne 2
2.3 Composants de la carte STM32F4 Discovery 3
2.4 Les ports d'entrées/sorties (GPIO) 4
3. Environnement de développement Keil µVision 5
3.1 Rôle de Keil, des packs et de CMSIS 5
3.2 Création du projet et choix du microcontrôleur 5
3.3 Installation du pack STM32F4 7
3.4 Configuration de l'environnement d'exécution 8
3.5 Ajout du fichier source 10
4. Programme réalisé 10
4.1 Code source 10
4.2 Explication du programme 11
4.3 Calcul des masques de bits 11
5. Problème rencontré et solution 12
5.1 Description du problème 12
5.2 Analyse de la cause 12
5.3 Solution appliquée par notre camarade 13
5.4 Solution retenue 13
6. Test du programme en simulation 14
7. Conclusion 14
Remerciements 14

1. Introduction et objectif

Un système embarqué est un ensemble électronique et informatique intégré dans un équipement afin d'en assurer une fonction précise : commande d'un moteur, lecture de capteurs, affichage, communication. Il est construit autour d'un microcontrôleur, c'est-à-dire un circuit intégré qui regroupe sur une même puce un processeur, de la mémoire et des périphériques d'entrées/sorties.

Ce TP a pour objectif de découvrir la famille de microcontrôleurs STM32 de STMicroelectronics et l'environnement de développement Keil µVision. Il s'agit de créer un projet pour le microcontrôleur STM32F407VG, de configurer correctement les fichiers de démarrage, puis d'écrire un premier programme en langage C qui commande la broche PA5 en accédant directement aux registres du microcontrôleur, sans générateur de code.

2. Présentation de la carte STM32

2.1 Le microcontrôleur STM32F407VG

Le STM32F407VG appartient à la série STM32F4, destinée aux applications nécessitant à la fois des performances de calcul élevées et une faible consommation. Son nom se décompose ainsi : STM32 (famille de microcontrôleurs 32 bits), F4 (série haute performance à cœur Cortex-M4), 07 (ligne de produits), V (boîtier de 100 broches) et G (1 Mo de mémoire Flash).

Caractéristique	Valeur
Cœur	ARM Cortex-M4, 32 bits, avec unité de calcul flottant (FPU) et instructions DSP
Fréquence maximale	168 MHz
Mémoire Flash	1 Mo, pour stocker le programme
Mémoire SRAM	192 Ko, pour les variables pendant l'exécution
Boîtier	LQFP100 (100 broches), dont jusqu'à 82 entrées/sorties
Ports GPIO	GPIOA à GPIOE utilisables sur ce boîtier, 16 broches par port
Périphériques	Timers, USART/UART, SPI, I2C, ADC 12 bits, DAC, USB OTG, CAN, Ethernet, DMA
Alimentation	1,8 V à 3,6 V (3,3 V sur la carte)

2.2 Architecture interne

La figure suivante présente l'organisation simplifiée du microcontrôleur. Le cœur Cortex-M4 exécute les instructions du programme stocké en Flash et manipule les données en SRAM. Une matrice de bus relie le cœur aux mémoires et aux périphériques, eux-mêmes répartis sur trois bus : AHB1 pour les ports GPIO et le DMA, APB1 et APB2 pour les périphériques de communication, les timers et les convertisseurs.

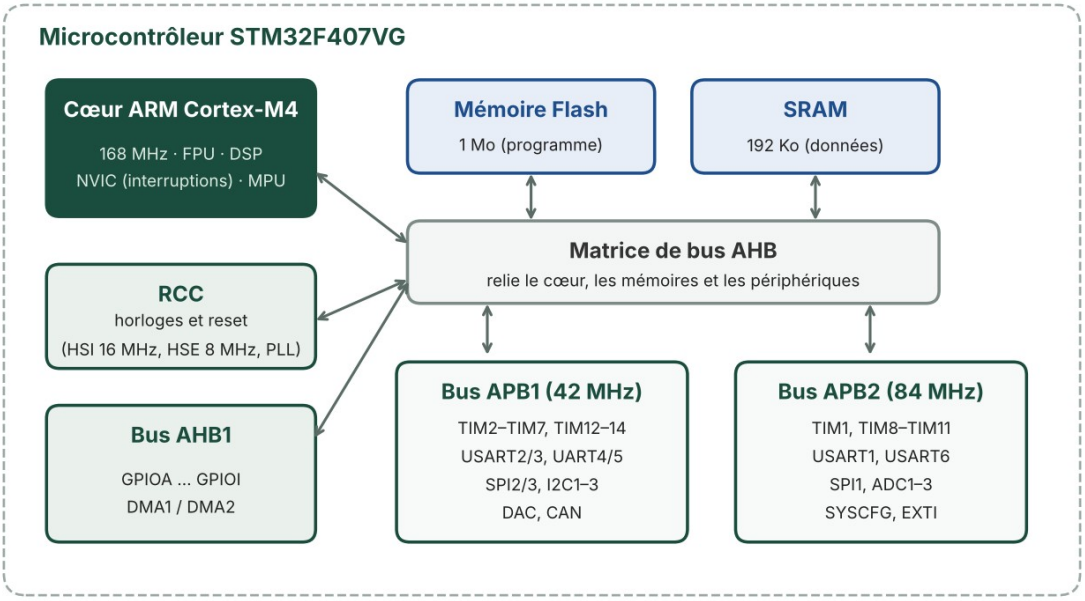


Figure 1 – *Architecture simplifiée du microcontrôleur STM32F407VG*

Le bloc **RCC** (Reset and Clock Control) joue un rôle central : il distribue l'horloge à chaque périphérique. Au démarrage, la plupart des horloges sont coupées afin de réduire la consommation. Avant d'utiliser un périphérique, il faut donc activer son horloge dans le registre correspondant du RCC ; c'est la première instruction de notre programme.

2.3 Composants de la carte STM32F4 Discovery

Le microcontrôleur est monté sur une carte de développement STM32F4 Discovery (référence STM32F407G-DISC1). Elle ajoute autour de lui les composants nécessaires à son alimentation, à sa programmation et à la réalisation d'expériences. La figure suivante repère les principaux composants ; leur rôle est détaillé dans le tableau qui suit.

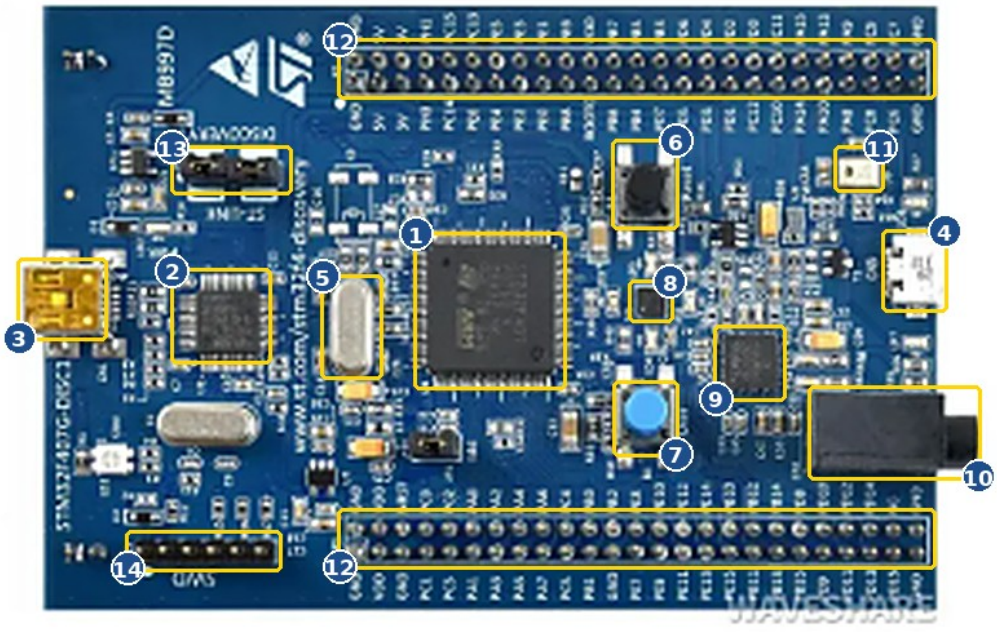


Figure 2 – *Principaux composants de la carte STM32F4 Discovery*

N°	Composant	Rôle
1	**Microcontrôleur STM32F407VGT6**	Cœur de la carte : il exécute le programme et pilote tous les périphériques. C'est lui que l'on programme dans ce TP.
2	**Programmateur ST-LINK/V2-A**	Second microcontrôleur, dédié à la liaison avec l'ordinateur. Il transfère le programme dans la Flash du STM32F407 et permet le débogage (pas à pas, points d'arrêt).
3	**Connecteur mini-USB (ST-LINK)**	Relie la carte à l'ordinateur : il fournit l'alimentation 5 V et la communication avec le ST-LINK.
4	**Connecteur micro-USB OTG**	Port USB propre au STM32F407, utilisable par les applications (clé USB, liaison avec un PC).
5	**Quartz 8 MHz**	Source d'horloge externe précise (HSE). La PLL du microcontrôleur la multiplie pour atteindre 168 MHz.
6	**Bouton RESET (noir)**	Redémarre le microcontrôleur : le programme reprend depuis le début.
7	**Bouton utilisateur USER (bleu)**	Bouton-poussoir relié à la broche PA0, utilisable comme entrée dans les programmes.
8	**Accéléromètre MEMS**	Capteur de mouvement sur trois axes, relié au microcontrôleur par le bus SPI.
9	**Convertisseur audio (DAC)**	Circuit CS43L22 qui transforme les données numériques en signal audio.
10	**Prise jack 3,5 mm**	Sortie audio pour brancher un casque ou un haut-parleur.
11	**Microphone numérique MEMS**	Capte le son et le transmet au microcontrôleur sous forme numérique.
12	**Connecteurs P1 et P2**	Deux rangées de broches qui donnent accès aux GPIO (PA0, PA5, PD12…) pour brancher des composants externes : LED, capteurs, afficheurs.
13	**Cavaliers CN3 (ST-LINK / DISCOVERY)**	Mis en place, ils relient le ST-LINK au STM32F407 de la carte. Retirés, le ST-LINK peut programmer une carte externe.
14	**Connecteur SWD (CN2)**	Sortie du ST-LINK pour programmer un autre microcontrôleur par l'interface SWD.

La carte comporte aussi un régulateur qui abaisse le 5 V de l'USB à 3,3 V, deux LED d'état (LD1 pour la communication ST-LINK, LD2 pour l'alimentation) et quatre LED utilisateur LD3 à LD6 (orange, verte, rouge et bleue) reliées aux broches PD13, PD12, PD14 et PD15.

2.4 Les ports d'entrées/sorties (GPIO)

Les broches du microcontrôleur sont regroupées en ports de 16 broches, nommés GPIOA, GPIOB, GPIOC, etc. La broche PA5 est ainsi la broche numéro 5 du port A. Chaque port est piloté par un ensemble de registres de 32 bits :

Registre	Rôle
`MODER`	Mode de chaque broche, codé sur 2 bits : 00 entrée, 01 sortie, 10 fonction alternative, 11 analogique.
`OTYPER`	Type de sortie : push-pull (0) ou drain ouvert (1).
`OSPEEDR`	Vitesse de commutation de la sortie.

Registre	Rôle
`PUPDR`	Résistances de tirage internes : aucune, pull-up ou pull-down.
`IDR`	Lecture de l'état des broches configurées en entrée.
`ODR`	Écriture de l'état des broches configurées en sortie : 1 niveau haut (3,3 V), 0 niveau bas (0 V).
`BSRR`	Mise à 1 ou à 0 d'une broche en une seule écriture, sans lecture préalable.

Après un reset, les broches sont configurées en entrée et les registres OTYPER, OSPEEDR et PUPDR gardent leur valeur par défaut (push-pull, vitesse basse, sans tirage), ce qui convient pour commander une LED. Le programme n'a donc besoin de modifier que MODER puis ODR.

3. Environnement de développement Keil µVision

3.1 Rôle de Keil, des packs et de CMSIS

Keil µVision (MDK-ARM) est un environnement de développement intégré qui regroupe un éditeur de code, le compilateur ARM, un simulateur et un débogueur. La prise en charge de chaque famille de microcontrôleurs est fournie sous forme de **packs**. Pour le STM32F407, il s'agit du pack **Keil::STM32F4xx_DFP** (Device Family Pack), qui contient la description du microcontrôleur, les fichiers de démarrage et les algorithmes de programmation de la Flash.

Le standard **CMSIS** (Cortex Microcontroller Software Interface Standard), défini par ARM, fournit une interface commune à tous les microcontrôleurs Cortex-M. Le fichier `stm32f407xx.h`, inclus au début du programme, s'appuie sur CMSIS : il donne un nom à chaque registre (`RCC->AHB1ENR`, `GPIOA->MODER`…) et à chaque bit, ce qui évite de manipuler des adresses numériques.

3.2 Création du projet et choix du microcontrôleur

Au lancement de Keil, la fenêtre principale comporte le menu, la fenêtre Project qui affiche l'arborescence du projet, et la fenêtre Build Output où apparaissent les messages de compilation.

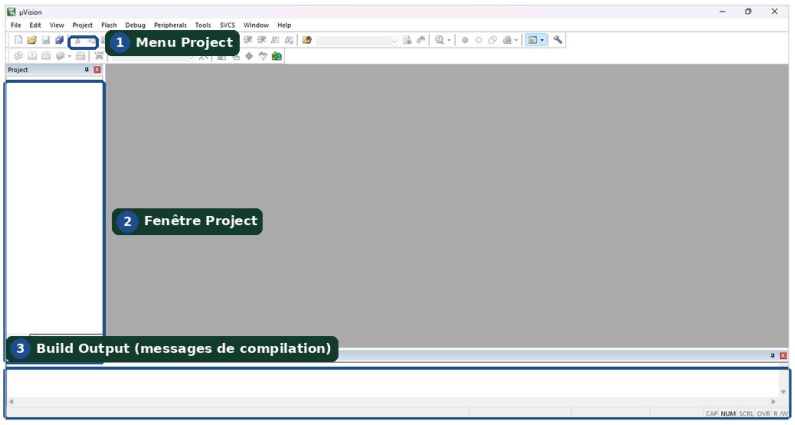


Figure 3 – *Fenêtre principale de Keil µVision*

Le projet est créé à partir du menu **Project → New µVision Project**, puis enregistré dans un dossier dédié.

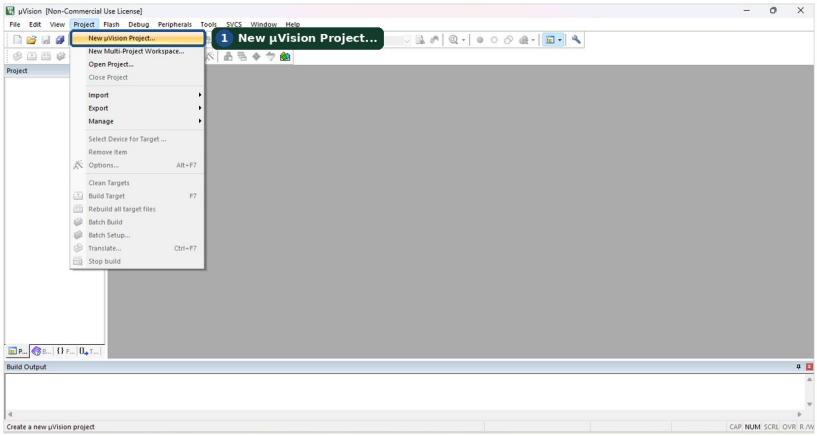


Figure 4 – *Création d'un nouveau projet*

Keil demande ensuite de choisir le microcontrôleur cible. Une fois le pack STM32F4 installé, le fabricant **STMicroelectronics** apparaît avec la série **STM32F4 Series**. Nous avons sélectionné le **STM32F407VGTx**. La zone Description confirme ses caractéristiques : cœur Cortex-M4 avec FPU et DSP.

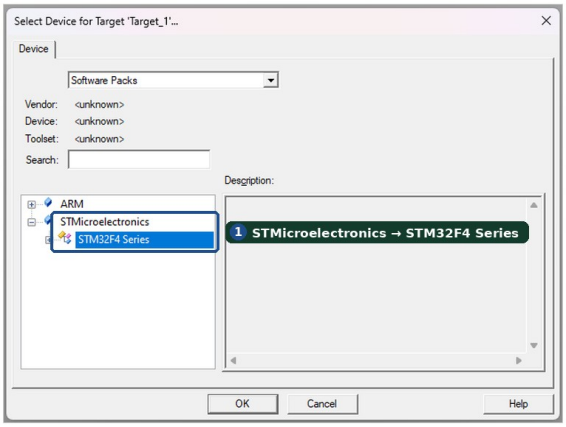


Figure 5 – *Fabricant et série disponibles après installation du pack*

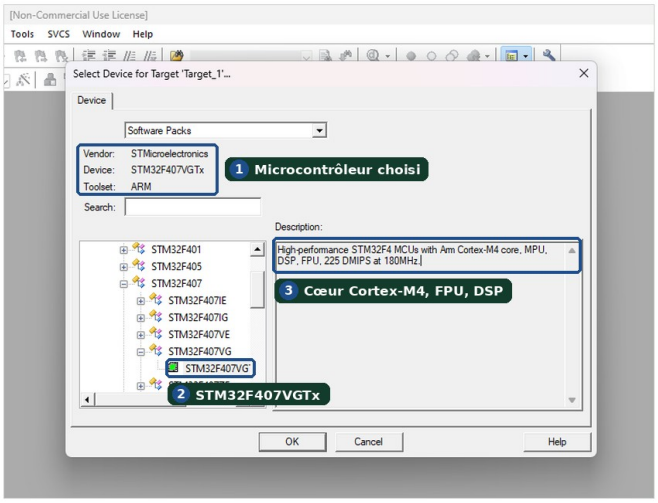


Figure 6 – *Sélection du microcontrôleur STM32F407VGTx*

3.3 Installation du pack STM32F4

Lors de la première configuration, Keil avait installé la version la plus récente du pack (**3.1.1**). Dans cette version, STMicroelectronics a retiré les fichiers de démarrage et la bibliothèque HAL : l'environnement d'exécution ne propose plus que le composant **CubeMX**, qui impose l'utilisation du logiciel externe STM32CubeMX. Or ce TP doit être réalisé uniquement avec Keil.

Nous avons donc ouvert le **Pack Installer**, sélectionné le STM32F407VGTx, puis installé l'ancienne version **2.17.1** depuis la rubrique Previous.

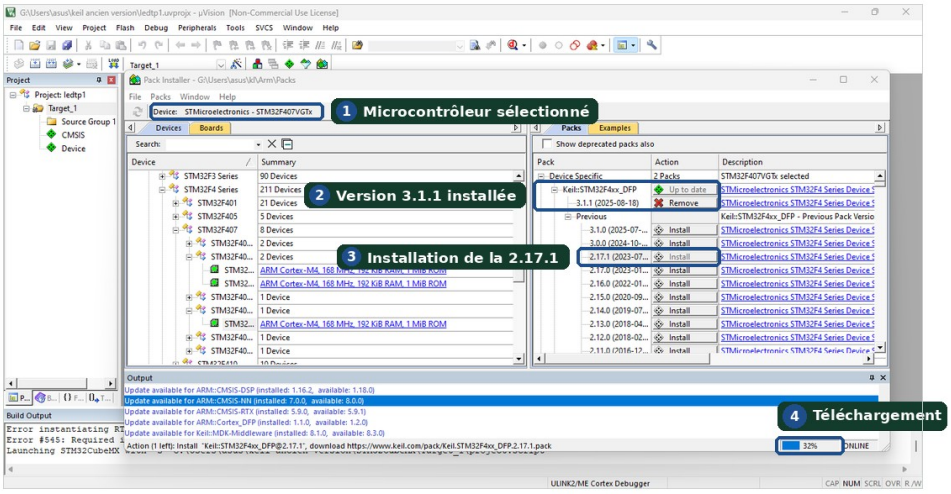


Figure 7 – *Installation de la version 2.17.1 du pack depuis le Pack Installer*

Une fois la version 2.17.1 installée, la version 3.1.1 a été désinstallée afin que Keil utilise uniquement l'ancienne version pour les nouveaux projets.

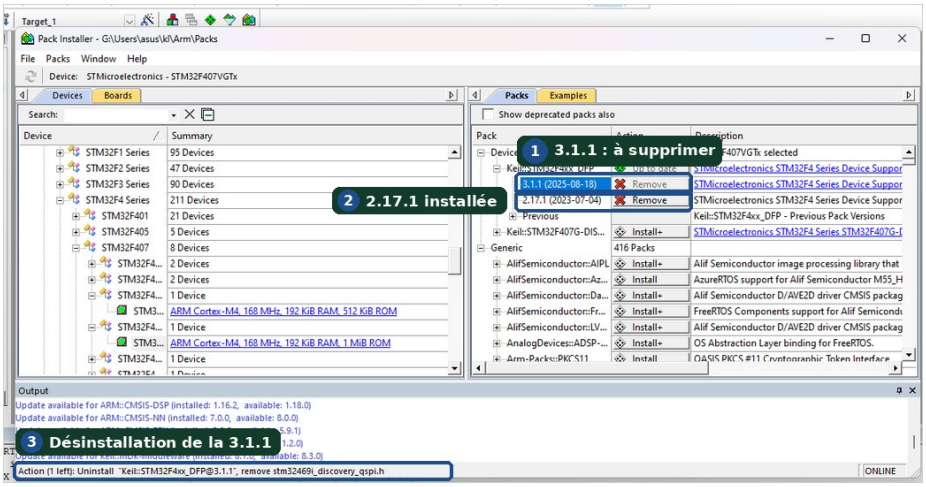


Figure 8 – *Désinstallation de la version 3.1.1*

3.4 Configuration de l'environnement d'exécution

La fenêtre **Manage Run-Time Environment** permet de choisir les composants logiciels ajoutés au projet. Avec le pack 3.1.1, la rubrique Device ne contenait aucun composant de démarrage, puis seulement CubeMX :

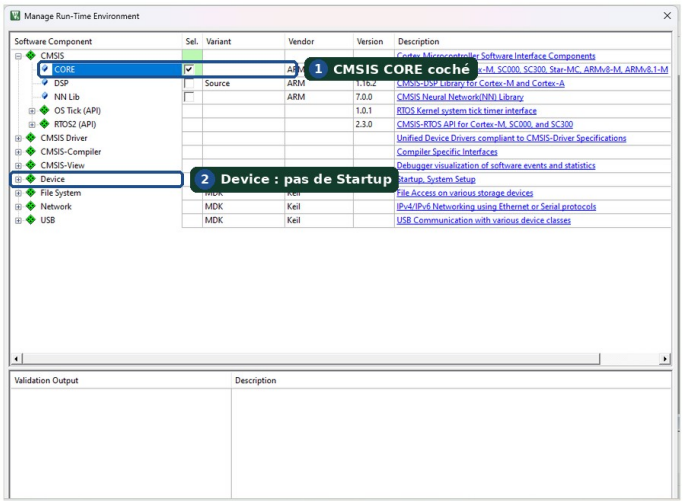


Figure 9 – *Environnement d'exécution sans composant Startup*

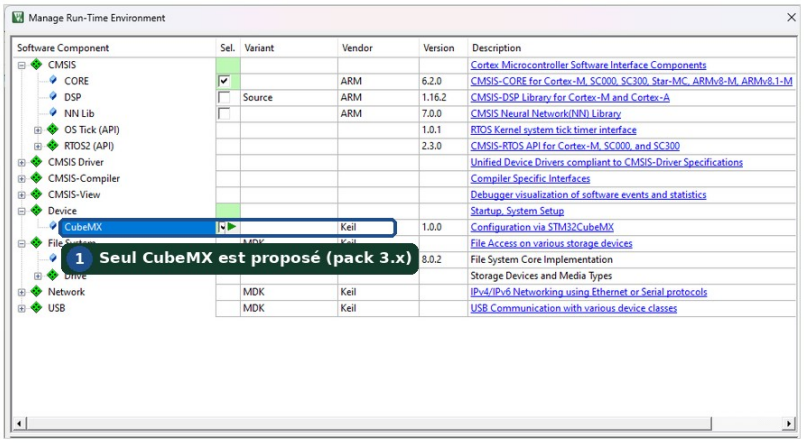


Figure 10 – *Seul le composant CubeMX est proposé avec le pack 3.1.1*

Avec le pack 2.17.1, le composant **Device → Startup** est disponible. Nous avons coché **CMSIS → CORE** et **Device → Startup**, puis validé avec Resolve et OK.

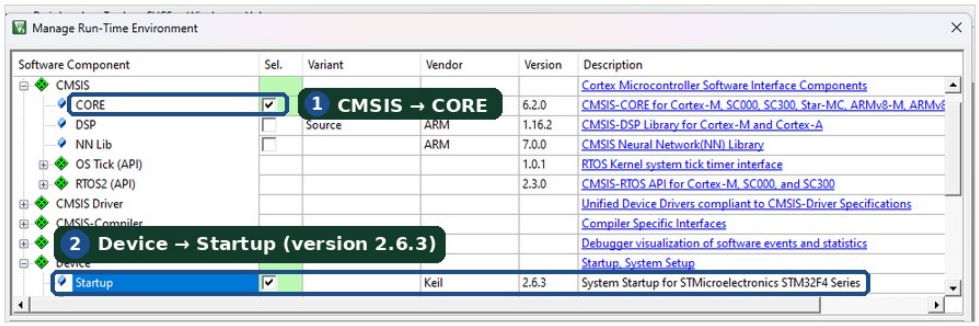


Figure 11 – *Composants CMSIS CORE et Device Startup sélectionnés*

Le tableau ci-dessous résume la différence entre les deux composants :

	Device → Startup	Device → CubeMX
Logiciel utilisé	Keil uniquement	Keil et STM32CubeMX
Fichiers ajoutés	startup_stm32f407xx.s et system_stm32f4xx.c	Code complet généré automatiquement (initialisations, HAL)
Configuration	Écrite par le programmeur dans le code	Réalisée graphiquement dans CubeMX
Intérêt pédagogique	Compréhension des registres et du démarrage	Rapidité de développement

Le fichier `startup_stm32f407xx.s`, écrit en assembleur, est le premier code exécuté après un reset : il initialise le pointeur de pile, contient la table des vecteurs d'interruption, appelle `SystemInit()` puis la fonction `main()`. Le fichier `system_stm32f4xx.c` contient `SystemInit()`, qui configure l'horloge du système.

3.5 Ajout du fichier source

Le projet contient désormais les groupes CMSIS et Device. Le fichier du programme est ajouté par un clic droit sur **Source Group 1** puis **Add New Item to Group**.

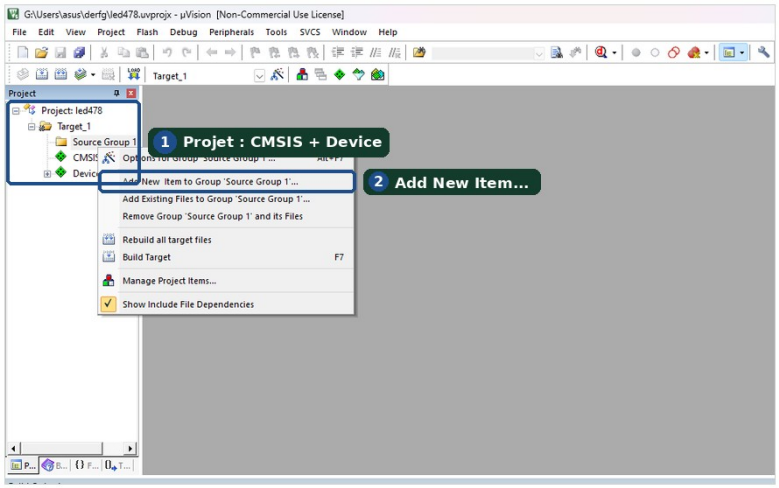


Figure 12 – *Ajout d'un nouvel élément au groupe Source Group 1*

Dans la fenêtre qui s'ouvre, nous avons choisi le type **C File (.c)**, nommé le fichier **main.c**, puis cliqué sur Add.

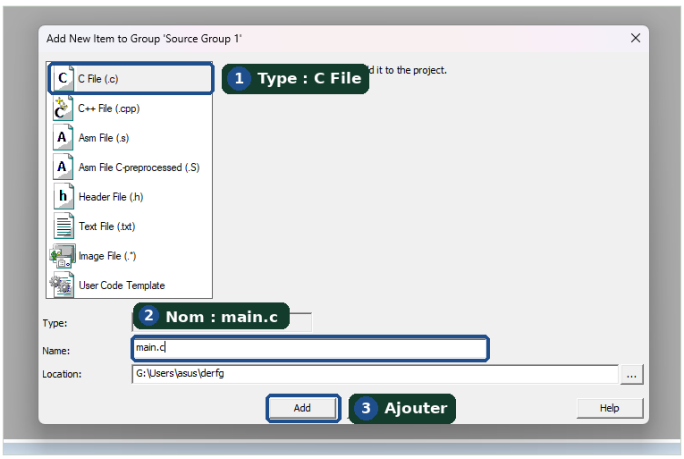


Figure 13 – *Création du fichier main.c*

4. Programme réalisé

4.1 Code source

Le programme demandé configure la broche PA5 en sortie puis inverse son état en boucle, ce qui fait clignoter une LED reliée à cette broche.

```
#include "stm32f407xx.h"

int main(void)
{
    /* Enable GPIOA clock */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

    /* Configure PA5 as output */
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |=  (1U << (5 * 2));

    while (1)
    {
        /* Toggle PA5 */
        GPIOA->ODR ^= (1U << 5);

        /* Small delay */
        for (volatile int i = 0; i < 10000; i++);
    }
}
```

4.2 Explication du programme

Instruction	Rôle
`#include "stm32f407xx.h"`	Inclut les définitions des registres et des bits du STM32F407.
`RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;`	Met à 1 le bit 0 du registre AHB1ENR pour activer l'horloge du port A. Sans cette horloge, les registres du port A ne réagissent pas.
`GPIOA->MODER &= ~(3U << (5 * 2));`	Met à 00 les bits 10 et 11 de MODER, qui correspondent à la broche PA5.
`GPIOA->MODER |= (1U << (5 * 2));`	Écrit 01 dans ces deux bits : PA5 est configurée en sortie.
`while (1)`	Boucle infinie : un programme embarqué ne se termine jamais.
`GPIOA->ODR ^= (1U << 5);`	Inverse le bit 5 de ODR grâce au OU exclusif : PA5 passe alternativement de 0 V à 3,3 V.
`for (volatile int i = 0; i < 10000; i++);`	Temporisation logicielle. Le mot-clé volatile empêche le compilateur de supprimer cette boucle vide lors de l'optimisation.

Les opérations « &= ~ » et « |= » permettent de modifier uniquement les bits de la broche PA5 sans changer la configuration des autres broches du port : c'est le principe du masquage de bits.

4.3 Calcul des masques de bits

Dans le registre MODER, chaque broche occupe 2 bits. La broche n occupe donc les bits 2n et 2n + 1. Pour PA5, il s'agit des bits 10 et 11, d'où le décalage de 5 × 2 = 10 positions.

Expression	Valeur binaire (bits 11 à 10)	Effet
`3U << 10`	11 (0x00000C00)	Masque des deux bits de PA5

Expression	Valeur binaire (bits 11 à 10)	Effet
`~(3U << 10)`	00 (0xFFFFF3FF)	Après le ET logique, les bits 10 et 11 sont remis à 0
`1U << 10`	01 (0x00000400)	Après le OU logique, PA5 est en mode sortie
`1U << 5`	bit 5 à 1 (0x00000020)	Après le OU exclusif sur ODR, l'état de PA5 est inversé

Avec une boucle de 10 000 itérations et l'horloge par défaut de 16 MHz, l'attente ne dure qu'environ un millième de seconde. La LED change donc d'état plusieurs centaines de fois par seconde et paraît allumée en permanence. Pour observer le clignotement à l'œil nu, il suffit d'augmenter la valeur, par exemple à 500 000.

5. Problème rencontré et solution

5.1 Description du problème

Lors de la séance, la compilation du projet de notre camarade a échoué avec le message « 1 Error(s) » et « Target not created ». La ligne `#include "stm32f407xx.h"` ainsi que toutes les lignes utilisant `RCC` et `GPIOA` étaient signalées par une croix rouge.

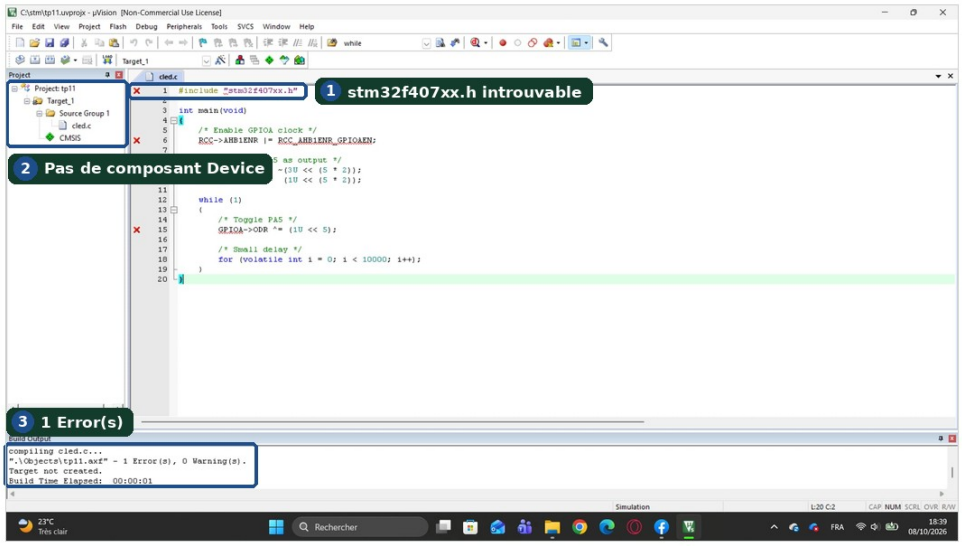


Figure 14 – *Erreurs de compilation dans le projet de notre camarade*

5.2 Analyse de la cause

L'arborescence du projet ne contenait que le groupe CMSIS : le composant **Device → Startup** manquait, car il n'existe pas dans la version 3.1.1 du pack STM32F4. Or c'est ce composant qui ajoute au projet le fichier `stm32f407xx.h` et le fichier de démarrage. Le compilateur ne trouvait donc pas ce fichier d'en-tête et ne connaissait ni `RCC` ni `GPIOA`. Le message complet, visible dans Build Output, est du type « cannot open source input file stm32f407xx.h ».

5.3 Solution appliquée par notre camarade

Notre camarade a coché le composant **Device → CubeMX** (figure 10), ce qui a généré un dossier STM32CubeMX dans le projet. Il a ensuite indiqué au compilateur le chemin du dossier contenant `stm32f407xx.h`, dans **Options for Target → C/C++ → Include Paths** :

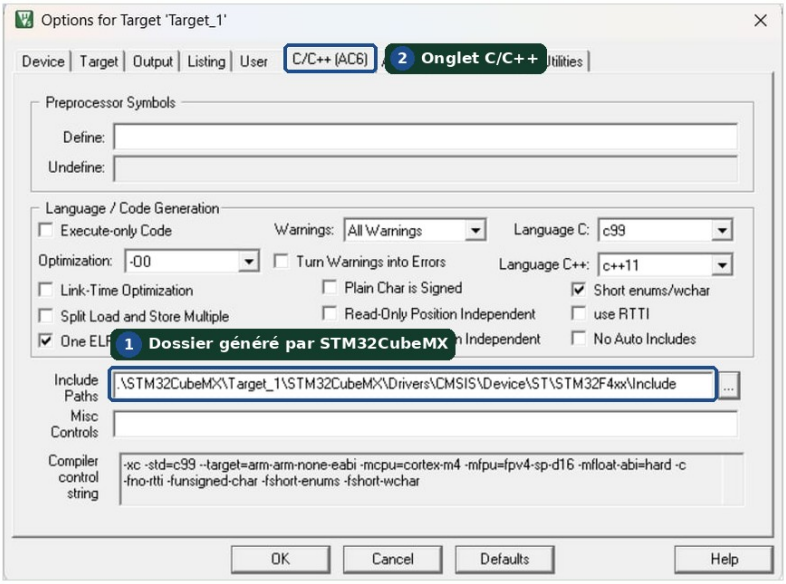


Figure 15 – *Ajout du chemin des fichiers générés dans Include Paths*

Cette correction supprime les erreurs, car le compilateur trouve désormais le fichier d'en-tête. Elle présente cependant deux limites : elle repose sur des fichiers produits par STM32CubeMX, alors que le TP doit être réalisé uniquement avec Keil, et le projet ne contient pas de fichier de démarrage (startup_stm32f407xx.s). Sans ce fichier, la table des vecteurs d'interruption et la fonction Reset_Handler manquent, et le programme risque de ne pas s'exécuter sur la carte réelle.

5.4 Solution retenue

Nous avons donc retenu une solution conforme aux consignes : installer la version 2.17.1 du pack STM32F4, désinstaller la version 3.1.1, puis sélectionner CMSIS → CORE et Device → Startup dans l'environnement d'exécution (paragraphes 3.3 et 3.4). Le fichier d'en-tête et le fichier de démarrage sont alors ajoutés automatiquement, sans modifier les Include Paths, et la compilation se termine sans erreur.

	Solution du camarade	Solution retenue
Origine des fichiers	Générés par STM32CubeMX	Fournis par le pack Keil 2.17.1
Réglage nécessaire	Chemin ajouté à la main dans Include Paths	Aucun : Device → Startup suffit
Fichier de démarrage	Absent	Ajouté automatiquement
Conforme au TP (Keil uniquement)	Non	Oui

6. Test du programme en simulation

En l'absence de la carte, le programme a été testé avec le simulateur intégré de Keil. Dans **Options for Target → Debug**, l'option **Use Simulator** a été sélectionnée. Le simulateur ne modélise pas tous les périphériques du STM32F407 : un fichier d'initialisation `sim.ini` a donc été ajouté afin d'autoriser l'accès à la zone mémoire des périphériques :

```
MAP 0x40000000, 0x5FFFFFFF READ WRITE
```

La session de débogage est lancée avec **Debug → Start/Stop Debug Session**. La fenêtre **Peripherals → System Viewer → GPIO → GPIOA** affiche le contenu des registres du port A. En exécutant le programme pas à pas (touche F10), on observe successivement :

- l'activation de l'horloge du port A après l'écriture dans RCC->AHB1ENR ;
- le champ MODER5 qui prend la valeur 01 : PA5 est configurée en sortie ;
- le bit ODR5 qui passe de 0 à 1 puis de 1 à 0 à chaque passage sur l'instruction GPIOA->ODR ^= (1U << 5).

Un point d'arrêt placé sur l'instruction d'inversion permet de vérifier, à chaque appui sur F5, que l'état de la broche change bien. Ce comportement correspond au clignotement attendu de la LED sur la carte réelle. Pour le téléversement sur la carte, il suffit de sélectionner ST-Link Debugger dans le même onglet et d'utiliser la commande Download (F8).

7. Conclusion

Ce TP nous a permis de découvrir le microcontrôleur STM32F407VG, son architecture et les composants de la carte STM32F4 Discovery. Nous avons appris à créer un projet dans Keil µVision, à choisir le microcontrôleur et à gérer les versions du pack STM32F4 afin de disposer des fichiers de démarrage sans recourir à STM32CubeMX.

Le programme réalisé montre les trois étapes indispensables pour commander une sortie en accès direct aux registres : activation de l'horloge du port dans le RCC, configuration de la broche dans MODER, puis écriture dans ODR. Le problème rencontré lors de la compilation nous a également fait comprendre le rôle du fichier d'en-tête du microcontrôleur et du fichier de démarrage. Ces bases serviront aux prochains TP, qui porteront sur les entrées, les interruptions et les timers.

Remerciements

Nous remercions M. Atef Ftirich, notre enseignant de TP, pour ses explications claires, ses conseils et sa disponibilité tout au long de cette séance. Nous remercions également l'établissement pour la mise à disposition du matériel et des logiciels nécessaires à la réalisation de ce travail.